

Experiment # 02

Implementation of Conditional Statements

1.1. Objective

Students will overview conditional statements including, switch-case statements and if-else.

1.2. Software

- Microsoft Visual Studio
- Dev C++

1.3. Theory

1.3.1.1. Conditional Statement

The conditional statements are used to execute a set of statements after testing a condition. The conditional statements are also called selection statements

1.3.1.2. The “if” Statement

The “**if statement**” evaluates a condition. If the given condition is *true*, the statement (or set of statements) following the “**if statement**” is executed. But if the given condition is *false*, the statement (or set of statements) following the “if statement” is ignored and the control transfers to the next statement.

Syntax of the “if Statement” is:

```
If (condition)
Statement – 1;
Statement – 2;
```

To execute a set of statements following the “**if Statement**”, the set of statements is enclosed in curly braces.

```
If (condition)
{
Statement – 1;
Statement – 2;
Statement – m
}
Statement – n
```

Example Code

```
int main()
{
int a;
cout<<"\n Enter the number = ";
cin>>a;
if (a%3==0)
{
cout<<" The number is divisible by 3";
```

```

}

cout<<"\n The number is not divisible by 3";
}

```

1.3.1.3. The “if..else” Statement

In this statement, one condition and two blocks of statements are given. Either one of the two blocks of statements is executed after evaluating a condition. If the given condition is **true**, the first block of statements is executed. But, If the given condition is **false**, the first block of statements is ignored and the second block following the else is executed.

Syntax of the “if else Statement”

```

if (condition)
    Statement – 1;
else
    Statement – 2;

```

In the case of multiple statements, the syntax is written as:

```

if (condition)
{
    Statement – 1;
    Statement – 2;
    Statement – m;
}
else
    Statement 3;
    Statement 4;
    Statement n

```

Example Code

```

void main()
{
    int a;
    cout<<" Enter the number = ";
    cin>>a;
    if (a>100)
    cout<<"\n The number is greater than 100";
    else
    cout<<"\n The number is less than 100";
}

```

1.3.1.4. Nested If Statement

When an “if statement” is used within another “if statement”, it is called the “nested if statement”. The “nested if statement” is used for multi way decision-making. The syntax of “nested if statement” is:

```

if (condition-1)

```

```

{
if (condition-2)
{
Statement-2;
}
Statement-3;
}

```

Example Code

```

void main()
{
int a,b,c;
cout<<"Enter first Number = ";
cin>>a;
cout<<"\n Enter second Number ";
cin>>b;
cout<<"\n Enter third Number ";
cin>>c;
if(a>b){
    if(a>c){
        cout<<"\n\n Ist Number is greater ";
    }
    else
        cout<<"\n\n Third number is greater ";    }
else
    if (b>c){
        cout<<"\n\n Second number is greater ";    }
    else
        cout<<"\n\n Third number is greater ";    }
}

```

1.3.1.5. Switch Statement

A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each case.

Syntax:

The syntax for a switch statement in C++ is as follows –

```
switch(expression) {
```

```
    case constant-expression :
```

```
        statement(s);
```

```
break; //optional
```

```
case constant-expression :
```

```
    statement(s);
```

```
break; //optional
```

```
// you can have any number of case statements.
default : //Optional
    statement(s);
}
```

Example:

```
#include <iostream>
using namespace std;
int main(){
    int num=5;
    switch(num+2) {
        case 1:
            cout<<"Case1: Value is: "<<num<<endl;
        case 2:
            cout<<"Case2: Value is: "<<num<<endl;
        case 3:
            cout<<"Case3: Value is: "<<num<<endl;
        default:
            cout<<"Default: Value is: "<<num<<endl;
    }
    return 0;
}
```

Rules:

The following rules apply to a switch statement

- The **expression** used in a **switch** statement must have an integral or enumerated type, or be of a class type in which the class has a single conversion function to an integral or enumerated type.
- You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
- The **constant-expression** for a case must be the same data type as the variable in the switch, and it must be a constant or a literal.
- When the variable being switched on is equal to a case, the statements following that case will execute until a **break** statement is reached.
- When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a break. If no break appears, the flow of control will *fall through* to subsequent cases until a break is reached.

- A **switch** statement can have an optional **default** case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No break is needed in the default case.

1.3.1.6. 3rd Method

```
#include <iostream>
using namespace std;
int main()
{
    int a=11,b=2;
    int c=b>a?b:a;
    cout<<c;

    return 0;
}
```

LAB TASKS

1. Create and Implement program in C++ to make a calculator that will Add, Subtract, Multiply and Divide the 2 integers (**USE IF ELSE**).
2. Create and Implement program in C++ Vowels. Make a method which displays entered character is a vowel or a consonant (**USE SWITCH CASE**).
3. Create and Implement program in C++ to check whether a number is odd or even (**USE SWITCH CASE**).